

Ecole Normale Supérieure de l'Enseignement Technique
Département Mathématique et informatique

Examen de rattrapage du 2^{ème} semestre 2024/2025

La note :.....	Date : 13/01/2025
Module : BIG DATA	Durée : 3h
Nom & Prénom :.....	Filière :.....

Instructions :

1. L'examen est composé de trois exercices, chacun portant sur une technologie spécifique.
2. Vous devez utiliser Docker pour installer et configurer chaque outil.
3. Répondez de manière claire et structurée. Incluez des captures d'écran ou des résultats de commandes lorsque cela est pertinent.

Enoncé: Gestion des Données pour une Entreprise de Commerce Électronique pour l'entreprise **ShopMaster**

I. Exercice 1 : HDFS

ShopMaster souhaite implémenter un système de stockage distribué pour gérer ses fichiers volumineux, tels que les images de produits et les rapports de ventes. Vous êtes chargé de configurer HDFS pour répondre à ce besoin.

1. Installez HDFS en utilisant Docker. Fournissez les commandes Docker utilisées.
2. Configurez un cluster HDFS simple avec un **NameNode** et deux **DataNodes**.
3. Importez un fichier texte contenant des informations de ventes dans HDFS. Le fichier doit être au format CSV, avec des colonnes `transaction\_id, user\_id, product\_id, timestamp, amount`. Montrez les commandes utilisées pour importer le fichier et vérifier son existence dans le système HDFS.

4. Lister le contenu du répertoire HDFS. Lire le début du fichier importé. Copier le fichier vers un autre répertoire dans HDFS. Fournissez les commandes et les sorties correspondantes.
5. Configurez la réplication du fichier importé avec un facteur de réplication de 3. Vérifiez et montrez que la réplication est correctement configurée.

II. Exercice 2 : Spring Batch

Une chaîne de supermarchés souhaite automatiser le traitement des données de ventes journalières pour analyser le chiffre d'affaires moyen par produit. Les données des ventes sont stockées dans un fichier CSV, et vous devez développer un job Spring Batch pour les traiter et générer un rapport.

Fichier d'entrée : sales.csv

TransactionID, Produit, Catégorie, Quantité, PrixUnitaire, DateVente

1,Pomme,Fruits,5,1.2,2025-01-01

2,Banane,Fruits,3,0.8,2025-01-01

3,Pain,Boulangerie,2,2.5,2025-01-01

4,Pomme,Fruits,4,1.2,2025-01-02

5,Lait,Laitages,1,1.0,2025-01-02

1. Développez un job Spring Batch qui lit les données depuis le fichier CSV, calcule le chiffre d'affaires pour chaque produit ($\text{Quantité} \times \text{PrixUnitaire}$), et stocke les résultats dans une base de données en mémoire (par exemple, H2).
2. Modifiez le job pour regrouper les données par catégorie et calculer le chiffre d'affaires total pour chaque catégorie.
3. Ajoutez une étape dans le job pour calculer le nombre de produits vendus par catégorie.
4. Ajoutez une étape dans le job pour calculer le chiffre d'affaires moyen par produit dans chaque catégorie.
5. Modifiez le job pour ignorer les enregistrements où l'un des champs Quantité ou PrixUnitaire est manquant ou si leurs valeurs sont inférieures ou égales à 0.
6. Modifiez le job pour qu'il soit planifié et s'exécute automatiquement tous les jours à minuit.

I. Exercice 3 : Kafka Streams

Une entreprise développant des véhicules autonomes souhaite analyser en temps réel les données envoyées par les véhicules pour surveiller leur performance, détecter les anomalies de conduite, et générer des alertes en cas de risque élevé d'accident. Les données des véhicules sont envoyées en continu via un topic Kafka.

Les messages sont envoyés sur un topic Kafka nommé `vehicle_data`. Chaque message est une chaîne de texte avec le format suivant :

<VehicleID>|<Speed>|<Latitude>|<Longitude>|<DistanceToNextObstacle>|<Timestamp>

Exemple :

1234|85|37.7749|-122.4194|10|2025-01-12T09:15:00Z

5678|45|40.7128|-74.0060|20|2025-01-12T09:16:00Z

9012|95|34.0522|-118.2437|8|2025-01-12T09:17:00Z

1. Développez une application Kafka Streams qui lit les données des véhicules depuis le topic Kafka **vehicle_data**, filtre les véhicules qui dépassent la vitesse autorisée (par exemple, 80 km/h), et génère un message d'alerte dans un topic Kafka nommé **speed_alerts**. Exemple de sortie dans le topic **speed_alerts** :

1234|85|37.7749|-122.4194|Overspeeding|2025-01-12T09:15:00Z

9012|95|34.0522|-118.2437|Overspeeding|2025-01-12T09:17:00Z

2. Modifiez l'application Kafka Streams pour détecter les véhicules qui ont une distance au prochain obstacle inférieure à un seuil critique (par exemple, 5 mètres) et publiez un message d'alerte dans un topic Kafka nommé **obstacle_alerts**.

Exemple de sortie dans le topic **obstacle_alerts** :

1234|3.5|Obstacle too close!

9012|2.8|Obstacle too close!